

**Prompt Injection Detection in Hindi, Hinglish, and English:
A Fairness Analysis and Layered Defense Framework**

A project report submitted in partial
fulfillment of the requirements for the degree of

Master of Science in Computer Science

By

Aishwarya Anand Tatapudi

University of Colorado Denver

May 2026

Project Advisor:

Professor Zheng Xiong Li

Department of Computer Science

University of Colorado Denver

ABSTRACT

This is a dual-pipeline prompt injection detection system across Hindi, Hinglish, and English. While the first pipeline fills an absence in multilingual prompt injection detection for Hindi and Hinglish using a hybrid MiniLM+SVM classifier with a reconstructed rule-based layer. The main result of this paper is the 12% FPR gap between Hindi (2.50% FPR) and Hinglish (15.00% FPR) users, the first per-language fairness study on the Srinivasan et al. (2026) dataset. An ablation study shows the gap is 2.5 percentage points wider with a training set composed of 65% English inputs. Pipeline 2 for English injection detection involved SPML Chatbot dataset's system and user prompts combined with a relevance gate and MiniLM+SVM classification model (+98.92% accuracy), compared to the zero-shot classifier using natural language inference (+56.84% accuracy), providing partial context-awareness. These results also show that task-specific training data is essential for good performance, especially for low-resource languages. Together, the results identify the need for language-specific architectures, context-sensitive detection strategies, and an evaluation approach that treats languages equally. While the rule layer provides marginal accuracy improvements (+0.25%), it does not reduce the fairness gap and slightly increases it from 12.00% to 12.50%, indicating limited effectiveness in multilingual settings.

This project report is approved as a recommendation to the Graduate committee.

Project Advisor:

Professor Zheng Xiong Li

Copyright © 2026 by **Aishwarya Anand Tatapudi** All Rights Reserved.

Table of Contents

1. Introduction.....	6
1.1 Problem Statement	6
1.2 Motivation for Model Choices.....	6
1.3 Project Contributions Summary.....	7
1.4 Organization of this Report.....	7
2. Background.....	8
2.1 Key Concepts	8
2.1.1 Prompt Injection Attacks	8
2.1.2 Code-Mixing and Hinglish	8
2.1.3 Sentence Encoders	8
2.1.4 Natural Language Inference (NLI)	8
2.1.5 Algorithmic Fairness.....	8
2.2 Related Work	9
3. Architecture.....	10
3.1 High Level Design	10
3.2 Pipeline 1 — Hindi/Hinglish Design	10
3.3 Pipeline 2 — English Design.....	10
4. Methodology, Results and Analysis	13
4.1 Pipeline 1 — Hindi/Hinglish	13
4.1.1 Datasets and Setup	13
4.1.2 Ablation Study	13
4.1.3 SVM vs Hybrid Results	13
4.1.4 Five-Tier Distribution	14
4.1.5 Fairness Gap Analysis.....	14
4.1.6 Comparison with Related Work.....	15
4.2 Pipeline 2 — English	15
4.2.1 Dataset and Setup.....	16
4.2.2 Experiment A — MiniLM + SVM Results	16
4.2.3 Experiment B — Zero-Shot NLI Results.....	16
4.2.4 Experiment A vs B Comparison	17
4.2.5 Relevance Gate Analysis	18
4.3 Cross-Pipeline Analysis	18
5. Conclusions.....	20

5.1 Summary	Error! Bookmark not defined.
5.2 Contributions.....	Error! Bookmark not defined.
5.3 Future Work.....	21
References	21

1. Introduction

1.1 Problem Statement

Popular Large Language Models (LLMs) like ChatGPT, Google's Gemini and Anthropic's Claude are already widely deployed in apps. As LLM deployment grows, prompt injection attacks, where a user crafts their input to override or add instructions to the LLM, have emerged as a risk. However, two important gaps in research exist.

The huge majority of detection research is conducted in English. Within India, hundreds of millions use AI systems in Hindi and Hinglish(Wang et al. 2024) , a code-mixed form of Hindi that uses Roman script, to send and share messages. Safety mechanisms only trained in English fail to generalize to these linguistic phenomena, and this problem is exacerbated by the large English-dominant population, which would be exposed to attacks out of the scope of existing tools. To date, there are no public datasets or defenses for prompt injection in Indian regional and code-mixed languages (Srinivasan et al. 2026).

Second, English-language detection systems only consider user prompts and do not consider the chatbot's deployment context. For example, a banking services chatbot and an educational services chatbot should act differently. While the system prompt defines the chatbot's intent and behavior, previous detector approaches ignore context. They fail to consider attacks that exploit the discrepancy between the system's original intent and user input.

1.2 Motivation for Model Choices

For efficient deployment, we use the pretrained sentence encoder MiniLM (paraphrase-multilingual-MiniLM-L12-v2), rather than fine-tuning a large transformer model like XLM-RoBERTa for downstream tasks that require sentence embeddings. MiniLM supports 50+ languages including Hindi, and runs efficiently even on standard CPUs. The 384-dimensional semantic embeddings can be computed within seconds. The accuracy of fine-tuned XLM-RoBERTa (99.70%; Srinivasan et al.) is better, but requires hours of training on GPUs. MiniLM+SVM achieves competitive accuracy while being considerably more efficient and thus is well suited for resource-constrained model deployment scenarios. MiniLM contains approximately 118 million parameters, compared to 270 million parameters in XLM-RoBERTa, making it more than twice as lightweight. In addition, training the SVM classifier completes in seconds on a CPU, whereas fine-tuning transformer models requires GPU resources and significantly longer training times.

Instead, we use a Support Vector Machine (SVM) with an RBF kernel as a classifier. SVMs are particularly attractive because they suffer no catastrophic forgetting, only need seconds to train on the embedding vectors, can be calibrated with Platt scaling to yield well-calibrated probability scores, and generalize well to small and medium datasets. The SVM is trained on top of fixed MiniLM embeddings, clearly separating representation learning from classification. This not only improves interpretability but also makes the pipeline more maintainable.

Pipeline 2 also applies zero-shot NLI models, which assess whether one sentence is entailed by another, to do the same task as the trained SVM classifiers, without requiring any

training examples. We investigate if zero-shot NLI can detect injection attacks, to provide a direct counter-example to the claim that labeled injection datasets are necessary for detection.

1.3 Project Contributions Summary

Contribution	Pipeline	Finding
First per-language fairness analysis on Srinivasan dataset	Pipeline 1	12% FPR gap Hindi vs Hinglish
Ablation study on English training data composition	Pipeline 1	65% English widens gap by 2.5%
Rule layer benign keyword redesign	Pipeline 1	Fixes FNR increase from generic keywords
Five-tier SOC confidence scoring system	Pipeline 1	92-93% prompts auto-handled
Root cause diagnosis of Hinglish fairness gap	Pipeline 1	Code-mixing embedding noise, not rule layer
Context-aware English detection with relevance gate	Pipeline 2	System+user prompt paired detection
MiniLM+SVM vs Zero-shot NLI comparison	Pipeline 2	Trained model 98.92% vs zero-shot 56.84%
Training data necessity proof	Pipeline 2	42% accuracy gap proves labeled data essential

Table 1: Summary of Contributions

1.4 Organization of this Report

Chapter 2 introduces background on prompt injection attacks, multilingual NLP, code-mixing, sentence encoders, NLI, and related work. Chapter 3 explains the dual-pipeline architecture and provides motivation for each design choice. Methodology and results of both pipelines are described in chapter 4 and contributions and future work in chapter 5.

2. Background

2.1 Key Concepts

2.1.1 Prompt Injection Attacks

Prompt injection attacks are adversarial prompts that cause an LLM to ignore its system-level prompts. Examples include: overriding commands, role switches, stealing information, gradually escalating prompting, framing as research done by other parties, and framing as role-play in a fictional scenario. Examples also include hiding prompts within other messages. These patterns may frequently be transliterated into other languages in multilingual contexts and are difficult for English-only detectors to recognize.

2.1.2 Code-Mixing and Hinglish

Code-mixing is interleaving two or more languages in a single utterance. Hinglish mixes Hindi vocabulary with Roman script transliteration and English syntax. This creates many interesting challenges for NLP. Hinglish does not conform to the orthographic rules of either parent language. Tokenizers for popular NLP frameworks do not apply well to Hinglish text. Pre-trained language models produce noisy embeddings. The embedding space of Hinglish is very ambiguous, since the embeddings for Hinglish lie between Hindi and English clusters in the embedding space, making the decision boundaries unreliable.

2.1.3 Sentence Encoders

A sentence encoder maps text to vectors of a fixed length such that similar sentences are embedded close to each other and dissimilar sentences far apart in the embeddings' space. MiniLM (also known as paraphrase-multilingual-MiniLM-L12-v2) is a multilingual sentence encoder with 118 million parameters. It was trained on sentence pairs in more than 50 languages, including an initial set of translations from English, and produces 384-dimensional output. It can run without a GPU. More importantly, MiniLM (paraphrase-multilingual-MiniLM-L12-v2) is trained using the Sentence-Transformers framework, which leverages NLI and paraphrase datasets to learn semantic similarity across sentences (Reimers and Gurevych, 2019). This enables the model to capture cross-lingual semantic relationships without requiring task-specific fine-tuning.

2.1.4 Natural Language Inference (NLI)

NLI predicts the entailment relation between a premise and a hypothesis among entailment, contradiction, or neutral, based on a pair of sentences. Models also learn topical and logical relations with low lexical overlap. NLI can also be used for zero-shot prompt injection detection, where the premise is the user prompt and the hypothesis is 'this is a prompt injection attack' by checking if the probability of entailment is higher than a chosen threshold. Because zero-shot NLI does not require labeled injection data, it can be especially useful for lower resource languages that may not have a large amount of labeled data.

2.1.5 Algorithmic Fairness

Algorithmic fairness means that the model should not perform systematically worse for speakers of some languages with respect to other languages. For injection detection, this means that the false positive rate should be similar across languages. If 15% of Hinglish speakers and 3% of Hindi speakers are incorrectly blocked, this linguistic bias in its safety system, irrespective of reporting a high aggregate accuracy, shows the need for measuring performance across groups to ensure equitable multilingual deployment.

2.2 Related Work

Rahman et al. (2024) extracted Multilingual BERT and DistilBERT embeddings from the deepset dataset, and used classical ML classifiers with an accuracy of 96.55% (no non-English or fairness evaluations). Chisty et al. (2024) used BERT+LSTM with GloVe embeddings, with 94.82% accuracy. Jain et al. (2025) compared twelve models and found that a neural RNN had 94.74% accuracy.

The first open Hindi/Hinglish prompt injection dataset was established by Srinivasan et al. (2026), who used a XLM-RoBERTa+rules ensemble system that achieved 99.70% on the overall accuracy. They did not report accuracies per language, just total accuracy over languages. A related work, conducted by Wang et al. (2024) to propose XSAFETY, claims that LLM safety performance considerably dropped for non-English languages.

In another work, Abbasi et al. (2025) introduced the SPML dataset and evaluated the performance of 19 models in English and Spanish. The work motivated using SPML for English-centric detection in Pipeline 2.

Our work is the first to perform per-language fairness analysis on Hindi/Hinglish and the first to analyze the impact of code-mixing on FPR disparities. We also compare classifier performance with zero-shot NLI models and with system prompt-aware context detection on English.

3. Architecture

3.1 High Level Design

The Hindi/Hinglish and English pipelines could be run independently because they had different linguistic issues to handle: the Hindi/Hinglish pipeline had to add a rule-based augmentation layer to handle code-mixing noise and script variation, while the English pipeline had to be adapted for context matching with the system prompts. Because both pipelines used the same MiniLM sentence encoder and SVM classifier, they could be compared.

Component	Pipeline 1 (Hindi/Hinglish)	Pipeline 2 (English)
Dataset	Srinivasan et al. 2026 + deepset	SPML Chatbot
Training size	3,746 prompts	5,552 prompts
Encoder	MiniLM (frozen, 384-dim)	MiniLM (frozen, 384-dim)
Classifier	SVM (RBF, C=1.0)	SVM (RBF, C=1.0)
Defense layer	Rule layer (28+48+39 patterns)	Relevance gate (cosine similarity)
Output	Five-tier confidence score	Three-way: DENY/INJECT/SAFE
Comparison model	N/A	Zero-shot NLI

Table 2: Pipeline Architecture Comparison

3.2 Pipeline 1 — Hindi/Hinglish Design

The first pipeline stage is the Embedding Layer, where each input is passed through the MiniLM model to obtain a 384-dimension semantic vector. It is frozen, with no fine-tuning on this injection task, mainly because of the CPU-only inference constraint, but also because the Hindi semantics are already captured from the multilingual pretraining of MiniLM.

The SVM Classifier (RBF kernel, C=1.0, class_weight=balanced) trains on the embedded representations from the Hindi+Hinglish+English combined training set. The weighting balances out the uneven label distribution in the training set. SVM is preferred to neural classifiers due to its lack of catastrophic forgetting, good generalization on medium-sized datasets and interpretable decision boundaries. Furthermore, tier assignment is based on calibrated SVM probabilities from Platt scaling.

The Rule Layer consists of a deterministic pattern matcher from Srinivasan et al. (2026) Section 7.3 that implements 28 triggers to override, 48 harmful keywords to reject, and 39 benign keywords to allow, returning Injection (hard block), SafeContext (hard allow), Suspicious (defer), or NoRule (defer) before the SVM. In practice, the rule layer activates for only 4–10% of prompts, with the majority of decisions deferred to the SVM classifier. One design decision was to use only specific multi-word benign phrases, rather than single token benign phrases, because SafeContext would otherwise filter out true attacks with such single tokens such as 'batao'.

Algorithm 1: Hybrid Prompt Injection Detection (Pipeline 1)

Input: Prompt P

Output: Classification (Injection or Safe) and Confidence Score

Step 1: Convert the prompt into a semantic vector using MiniLM

Step 2: Check if the prompt matches any rule-based patterns

Step 3: If the rule layer detects a clear attack

→ classify as Injection with confidence = 1.0

Step 4: If the rule layer detects a clearly safe context

→ classify as Safe with confidence = 0.0

Step 5: Otherwise (uncertain case), use the trained SVM model

→ compute the probability of being an attack

Step 6: If probability ≥ 0.5

→ classify as Injection

Else

→ classify as Safe

Step 7: Return to the final classification and confidence score

The Five-Tier Scoring maps SVM confidence to Critical ($\geq 90\%$, auto block), High ($\geq 70\%$, same-day CSR review), Medium ($\geq 50\%$, next-day review), Low ($\geq 30\%$, batch monitor) and Safe ($< 30\%$, auto allow). The SOC alignment enabled human-in-the-loop workflows that fit customers' deployment scenarios.

Algorithm 2: Five-Tier Confidence Scoring

Input: Attack probability p

Output: Risk Tier

1: if $p \geq 0.90$ then return Critical

2: else if $p \geq 0.70$ then return High

3: else if $p \geq 0.50$ then return Medium

4: else if $p \geq 0.30$ then return Low

5: else return Safe

3.3 Pipeline 2 — English Design

Pipeline 2 improves on this context-aware detection process. It leverages the corpus of paired system and user prompts collected in the SPML dataset. There is a difference between these two types of prompts. The system prompt provides context during deployment, including the relevance gating, while the user prompt includes the various potential attack patterns for SVM classification. User prompts were truncated to 512 tokens to match the input constraints of the NLI model and sentence encoder, which can result in minor loss of contextual information. A hypothesis template ("This text is a { }.") was used to convert the classification task into an entailment problem.

Algorithm 3: English Prompt Injection Detection (MiniLM + SVM)

Input: User Prompt U

Output: Label (Injection / Safe) Confidence Score

```
1:  $v \leftarrow \text{MiniLM\_Encode}(U)$ 
2:  $p \leftarrow \text{SVM.predict\_proba}(v)$ 
3: if  $p \geq 0.5$  then
4: return Injection, p
5: else
6: return Safe, p
7: end if
```

The Relevance Gate checks the cosine similarity between the system and user message vectors. If similarity falls below a certain threshold, the output is DENY_IRRELEVANT, indicating the user message lies outside the bot's scope. If no system prompt is present, the gate is toggled for a general assistant, with threshold optimization tested at values ranging from 0.10 to 0.50.

In Zero-shot NLI Comparison, a cross-encoder/nli-deberta-v3-small model is compared to the trained SVM. The cross-encoder produces a probability whether there is an entailment between an user's prompt (the premise) and the hypothesis "prompt injection attack". As it does not need training data, it tests whether general NLI understanding can replace training. A threshold of 0.5 was used to classify prompts as injection or safe based on the NLI score. In cases where the NLI model failed to produce a prediction, a default probability of 0.5 was assigned to avoid biasing the classification.

Algorithm 4: Zero-Shot NLI Prompt Injection Detection

Input: User Prompt U

Output: Label (Injection / Safe) Confidence Score

```
1: Define labels  $L = [\text{"prompt injection attack"}, \text{"safe user message"}]$ 
```

```

2: result  $\leftarrow$  NLI_Model (U, L)
3: p  $\leftarrow$  probability to classify as "prompt injection attack"
4: if p  $\geq$  0.5 then
5: return Injection, p
6: else
7: return Safe, p
8: end if

```

4. Methodology, Results and Analysis

4.1 Pipeline 1 — Hindi/Hinglish

4.1.1 Datasets and Setup

The primary training data are from Srinivasan et al. (2026) 4000 prompts (2000 Hindi & 2000 Hinglish) also balanced to 50% safe/injection prompts. For training only, we add an additional English dataset (deepset/prompt-injections, 546 prompts). The data was split for each language into 80% training and 20% test prompts. All experiments were run with `random_state=42`.

4.1.2 Ablation Study

Three training compositions were evaluated to determine the effect of English data augmentation on the fairness gap:

Model	Training Data	Size	Hindi Acc.	Hinglish Acc.	Fairness Gap
Model 1	Hindi + Hinglish only	3,200	95.75%	89.00%	11.50%
Model 2	+ deepset English (15%)	3,746	96.75%	89.00%	12.00%
Model 3	+ deepset + SPML (65%)	10,686	96.50%	86.25%	14.50%

Table 3: Ablation Study — Effect of English Training Data Composition

Model 2 performs best. It also achieves a 1% increase in Hindi accuracy (15% of the training data is with 546 deepset prompts) without sacrificing Hinglish, because MiniLM's cross-lingual embeddings semantically map attacks across languages. For model 3, 65% English augmentation degrades Hinglish accuracy by 2.75 percentage points while also increasing the fairness gap by 3 percentage points, thus showing the harm of indiscriminate use of English data augmentation for Hinglish speakers.

4.1.3 SVM vs Hybrid Results

Metric	SVM Only	Hybrid (Rule+SVM)	Change
Hindi Accuracy	96.75%	97.00%	+0.25%
Hindi FPR	3.00%	2.50%	-0.50%
Hindi FNR	3.50%	3.50%	0.00%

Metric	SVM Only	Hybrid (Rule+SVM)	Change
Hinglish Accuracy	89.00%	89.25%	+0.25%
Hinglish FPR	15.00%	15.00%	0.00%
Hinglish FNR	7.00%	6.50%	-0.50%
Combined Accuracy	92.88%	93.12%	+0.24%
Fairness Gap (FPR)	12.00%	12.50%	+0.50%

Table 4: Pipeline 1 Results — SVM Only vs. Hybrid Model

For Hindi, the rule layer catches 18 attacks (4.5%), while allowing through 25 innocent prompts (6.2%). The rule layer catches 15 Hinglish attacks (3.8%). All 22 false positive (FPR) Hinglish predictions are Rule: NoRule. This means that the layer never fires on the Hinglish dataset, and the SVM boundary of the Hinglish FPR problem is completely due to code-mixing noise, not a rule layer weakness. Although the rule layer improves overall accuracy slightly, it does not reduce the fairness gap. The gap increases from 12.00% to 12.50% because the rule layer triggers less frequently for Hinglish prompts (3.8%) compared to Hindi (4.5%), leaving most Hinglish predictions to the SVM classifier.

4.1.4 Five-Tier Distribution

Tier	Threshold	Hindi	Hindi %	Hinglish	Hinglish %	Action
Critical	$\geq 90\%$	175	43.8%	177	44.2%	Auto block
High	$\geq 70\%$	14	3.5%	20	5.0%	Same-day CSR
Medium	$\geq 50\%$	11	2.8%	20	5.0%	Next-day CSR
Low	$\geq 30\%$	6	1.5%	21	5.2%	Batch monitor
Safe	$< 30\%$	194	48.5%	162	40.5%	Auto allow

Table 5: Five-Tier Confidence Distribution (400 prompts per language)

Among those auto-handled, 92.3% of Hindi and 84.7% Hinglish prompts are auto-handled. Hinglish also has a considerably longer CSR queue (10.0%, compared to 6.2% for Hindi), which suggests operational fairness: Hinglish users are at a disproportionately higher risk of being sent to human review because of classification uncertainty introduced by the code-mixing noise. Hinglish prompts are more frequently assigned to High and Medium tiers, increasing human review requirements and reflecting higher uncertainty in classification.

4.1.5 Fairness Gap Analysis

Metric	Hindi	Hinglish	Gap
False Positives (wrongly blocked)	3	22	19 more for Hinglish
False Negatives (attacks missed)	6	13	7 more for Hinglish
CSR queue (High + Medium)	25	40	15 more for Hinglish
FPR	2.50%	15.00%	12.50% disparity
FNR	3.50%	6.50%	3.00% disparity

Table 6: Fairness Gap Summary — Hindi vs Hinglish

The 12% difference in FPR between Hinglish and Hindi means Hinglish users are falsely blocked five times as often as Hindi users, simply because they are writing in Hinglish rather than Hindi. All 22 Hinglish false positives were innocuous (e.g., queries about education, requests for poetry, and questions about professional planning). All 22 are mapped to a partially overlapping embedding region by MiniLM. This is despite the prompts themselves, not the labels from dataset creation, not causing this issue: MiniLM was not trained to view Hinglish as a separate language, and Hinglish vectors are placed in a noisy region between Hindi and English, where attack and innocent prompts are poorly separated. Additionally, the rule layer rarely activates for Hinglish prompts, as shown by the low rule-trigger rate (3.8%), meaning that most Hinglish predictions depend on the SVM decision boundary.

4.1.6 Comparison with Related Work

Model	Language	Accuracy
Srinivasan et al. (2026) XLM-RoBERTa fine-tuned + Rules	Hindi/Hinglish	99.70%
This work — Hybrid (MiniLM+SVM+Rules)	Hindi/Hinglish	93.12%
This work — SVM only (MiniLM+SVM)	Hindi/Hinglish	92.88%
Rahman et al. (2024) mBERT+LR	English	96.55%
Chisty et al. (2024) BERT+LSTM	English	94.82%
Jain et al. (2025) RNN	English	94.74%

Table 7: Comparison with State-of-the-Art Methods

Model	Parameters	Training Cost	Hardware	Training Time
MiniLM+SVM	~118M	No fine-tuning	CPU	Seconds
XLM-RoBERTa	~270M	Fine-tuned	GPU	Hours

This 6.58% error vs Srinivasan et al. is due to fine-tuning vs the frozen embedder. Srinivasan et al. fine-tune XLM-RoBERTa (ii) (270M parameters) on the task for hours on a GPU. This uses MiniLM, a frozen embedding, and a CPU-deployed SVM (which we can train in seconds). Compared to Srinivasan et al. (2026), the proposed method shows a 6.58% lower accuracy gap

relative to Srinivasan et al. is the trade-off for lightweight deployment. While their model uses a 270M parameter XLM-RoBERTa fine-tuned on GPUs over several hours, our pipeline uses an 118M parameter MiniLM encoder with no fine-tuning and a CPU-based SVM that trains in seconds. This demonstrates a significant reduction in computational costs while maintaining competitive performance. The rule layer contributes only marginally to performance improvements, indicating that most detection capability is derived from the SVM classifier rather than rule-based logic. The accuracy of Pipeline 1 is like all English-language baselines, despite being a more difficult multilingual task.

4.2 Pipeline 2 — English

4.2.1 Dataset and Setup

The SPML Chatbot Prompt Injection Dataset contains 16012 rows of system prompts and user prompts. Each row of the dataset contains a system prompt, specifying the role of the chatbot, and an user prompt. This pairwise setup enables contextual detection not possible with other English datasets. The validated dataset contained 6,940 prompts, balanced between 3,470 attacks and 3,470 innocent prompts, and was split into a training set of 5,552 prompts and a test set of 1388 prompts.

4.2.2 Experiment A — MiniLM + SVM Results

Metric	Value
Accuracy	98.92%
Precision	99.85%
Recall	97.98%
F1	98.91%
FPR	0.14%
FNR	2.02%
True Positives	680
True Negatives	693
False Positives	1
False Negatives	14

Table 8: Pipeline 2 Experiment A — MiniLM+SVM on SPML (1,388 test prompts)

MiniLM+SVM achieves 98.92% SPML English accuracy and beats Pipeline 1's 93.12% Hindi/Hinglish accuracy; user prompts for SPML English are phonetically more uniform and distinct in embedding space than Hinglish, which is prone to code-mixing. One innocent prompt is mis-classified; 14 attacks escape detection, so the model still very accurately blocks attacks. Classification was performed using only the user prompts, while system prompts were used separately for relevance analysis.

4.2.3 Experiment B — Zero-Shot NLI Results

Metric	Value	Interpretation
Accuracy	56.84%	Near random guessing (50% = coin flip)
Precision	100.00%	Every flagged prompt was a real attack
Recall	13.69%	Only caught 13.69% of attacks
F1	24.08%	Very poor overall performance
FPR	0.00%	Never wrongly blocked innocent users
FNR	86.31%	Missed 86.31% of attacks
True Positives	95	Attacks correctly caught
True Negatives	694	Innocent correctly allowed
False Positives	0	No innocent wrongly blocked
False Negatives	599	Attacks missed

Table 9: Pipeline 2 Experiment B — Zero-Shot NLI on SPML (1,388 test prompts)

Zero-shot NLI is extremely conservative, marking only 95 of 694 attacks as true, missing 599. The model achieves perfect precision and zero false positives, since it only flags prompts that it is totally certain the hypothesis 'prompt injection attack' is entailed. However, this is not a problem because most injection attacks do not use language related to the technical concept of prompt injection and will be classified as harmless by the general NLI model. This conservative behavior results from the fixed decision threshold (0.5) and the use of a strict entailment formulation.

4.2.4 Experiment A vs B Comparison

Metric	MiniLM+SVM (Trained)	Zero-shot NLI (No Training)	Winner
Accuracy	98.92%	56.84%	MiniLM+SVM (+42.08%)
Precision	99.85%	100.00%	Zero-shot NLI
Recall	97.98%	13.69%	MiniLM+SVM (+84.29%)
F1	98.91%	24.08%	MiniLM+SVM (+74.83%)
FPR	0.14%	0.00%	Zero-shot NLI
FNR	2.02%	86.31%	MiniLM+SVM (-84.29%)
Training required	Yes	No	Zero-shot NLI

Table 10: Pipeline 2 Comparison — MiniLM+SVM vs Zero-Shot NLI

The drop in accuracy from MiniLM+SVM training to zero-shot NLI (42.08%) shows that if we want reliable detection of injections, we need labeled examples to train the detector. However, the advantage of the zero-shot model is that it has perfect precision (and thus zero false positive rate). If it detects injection, then it's correct. That said, an FNR of 86.31% makes it patently unsuitable on its own as a detector, and for low-resource languages without labeled injection dataset's the performance of zero-shot methods is limited.

4.2.5 Relevance Gate Analysis

Nine thresholds were tested for the cosine similarity relevance gate from 0.10 to 0.50. The optimal value is 0.10, blocking 139 out of 157 total prompts, of which 18 are attacks and 121 are innocent prompts. Therefore, only 13% gate precision can be obtained. Overall accuracy drops from 98.92% to 90.35% and FPR increases from 0.14% to 17.87% on the same model.

This gate failure (caused by MiniLM) occurs because it is based on both lexical and surface similarity. In the example of a healthcare system prompt and an user asking about dehydration symptoms, they have low cosine similarity because they have low lexical similarity, though they are semantically similar. This would not confuse an NLI cross-encoder model into thinking it was entailment, as it would have unique token distributions. Using an NLI gate rather than cosine similarity is left as future work.

4.3 Cross-Pipeline Analysis

Metric	P1 Hindi	P1 Hinglish	P2 MiniLM+SVM	P2 Zero-shot NLI
Accuracy	97.00%	89.25%	98.92%	56.84%
FPR	2.50%	15.00%	0.14%	0.00%
FNR	3.50%	6.50%	2.02%	86.31%
Defense	Rule+SVM	Rule+SVM	SVM only	NLI only
Training	Required	Required	Required	Not required
Output	Five-tier	Five-tier	Binary	Binary

Table 11: Cross-Pipeline Performance Summary

Two results emerge. First, English detection (98.92%) is considerably better than Hinglish Detection (89.25%) for the same MiniLM+SVM model, suggesting that it is not model expressiveness that is missing, but rather code-mixed noise that affects Hinglish fairness gap. Second, classifiers trained on all recall-based metrics outperform zero-shot models, indicating that labeled injection datasets are architecturally required to achieve accurate classification on all languages of interest. For these two reasons, language-specific training data and task-specific labeled examples are irreplaceable for a reliable multilingual injection detection. Training time for the SVM classifier was measured to be under 10 seconds on a CPU, confirming the efficiency of the proposed pipeline.

5. Conclusions

5.1 Summary

Section	What Was Done	Key Finding
Pipeline 1 Ablation	Tested 3 training compositions	65% English widens Hinglish fairness gap by 2.5%
Pipeline 1 Hybrid	Added rule layer to SVM baseline	Rule layer provides marginal accuracy improvement but does not reduce fairness gap; effectiveness limited due to low activation on Hinglish inputs.
Pipeline 1 Fairness	Per-language FPR/FNR analysis	12% FPR gap — Hinglish blocked 5x more than Hindi
Pipeline 1 Five-Tier	SOC confidence scoring	92-93% prompts auto handled without human review
Pipeline 2 MiniLM+SVM	Trained on SPML English	98.92% accuracy, 0.14% FPR
Pipeline 2 Zero-shot NLI	No training, direct NLI classification	56.84% accuracy, 86.31% FNR
Pipeline 2 Gate	Cosine similarity relevance gate	Gate fails — NLI cross-encoder needed instead
Cross-pipeline	Compared all results	Training data essential; code-mixing causes fairness gap

Table 12: Project Summary

5.2 Contributions

- Perform the first per-language fairness analysis of Srinivasan et al.'s (2026) Hindi/Hinglish dataset, finding a 12% FPR disparity that would be hidden by aggregate reporting.
- Demonstrates a lightweight multilingual detection pipeline that reduces computational cost by over $2\times$ in model size and eliminates GPU-based fine-tuning while maintaining competitive accuracy
- An ablation study shows English training data accounts for 65% of all training data and increases the Hindi/Hinglish fairness gap by 2.5 percentage points.
- Empirical finding that rule-based defenses provide limited fairness improvement, with only 4–10% rule activation and no reduction in FPR disparity across languages
- Five-tier confidence scoring system mapped to SOC severity levels, allowing human-in-the-loop review workflows for multilingual LLM deployments.
- Root cause - Hinglish FPR gap and relevance gate failure stem from the same limitation of MiniLM embedding, prompting a unified recommendation for the architecture.
- Shows context-aware English detection pipeline with SPML paired system+user prompts, and partial context-awareness through relevance gating.

- Empirical comparison of a trained MiniLM+SVM (98.92%) with zero-shot NLI (56.84%) shows that labeled injection training data is essential. The results have a clear implication for low-resource language deployment.
- Extends the security-fairness tradeoff concept to guide threshold selection in multilingual injection detection systems for various deployments. Security-fairness tradeoff framework providing deployment-context-dependent threshold selection guidance for multilingual injection detection systems.

5.3 Future Work

Fine-tuning XLM-RoBERTa or a dedicated Hinglish model would close the fairness gap while simultaneously learning Hinglish-specific decision boundaries. Creating a dedicated Hinglish corpus with speaker verification would remove translation artifacts present in the Hinglish corpus used in this study. Replacing the cosine similarity gate with a cross-encoder NLI model (e.g. cross-encoder/nli-MiniLM2-L6-H768) would fix the relevance gate failure by making it attuned to topical entailment instead of surface-level similarity. With few-shot NLI (10-50 labeled injection examples), the gap between zero-shot (56.84%) and fully trained (98.92%) performance could be narrowed. Post-processing filtering of harmful output from LLMs post input filtering could add defense-in-depth for attacks that bypass input filtering. Extending this framework to Tamil, Telugu, Bengali, and Malayalam would address the broader multilingual safety gap in India.

References

- Abbasi, M., Hernandez, G., Prieto, J., & Corchado, J. M. (2025). Multilingual prompt injection attacks detection: Evaluating adversarial robustness in large language models. Proceedings of [Conference/Journal].
- Chisty, M. M. O., Sakib, A. N. M., Khan, M. S., & Shuvo, M. B. (2024). Prompt injection detection using ensemble of BERT and LSTM with GloVe embeddings. In Proceedings of the 13th International Conference on Electrical and Computer Engineering (ICECE). IEEE.
- Jain, B., Pawar, P., Gada, D., Patwa, T., Kanani, P., Patil, D., & Kurup, L. (2025). Comprehensive analysis of machine learning and deep learning models on prompt injection classification. International Research Journal of Multidisciplinary Technovation.
- Lin, H., Lao, Y., Geng, T., Yu, T., & Zhao, W. (2025). UniGuardian: A unified defense for detecting prompt injection, backdoor attacks and adversarial attacks in large language models.
- Rahman, M. A., Shahriar, H., Wu, F., & Cuzzocrea, A. (2024). Applying pre-trained multilingual BERT in embeddings for improved malicious prompt injection attacks detection. In Proceedings

of the 2024 International Conference on Artificial Intelligence, Blockchain, Internet of Things. IEEE.

Srinivasan, J., Regi, S. A., Anbarasan, A. K., Suresh, A., Vetriselvi, T., & Venu, S. (2026). Detection and analysis of prompt injection in Indian multilingual large language models. Scientific Reports.

Wang, W., Tu, Z., Chen, C., Yuan, Y., Huang, J., Jiao, W., & Lyu, M. R. (2024). All languages matter: On the multilingual safety of LLMs.

Reimers, N., & Gurevych, I. (2019). Sentence-BERT: Sentence embeddings using Siamese BERT-networks. Proceedings of EMNLP-IJCNLP.